Videos

Central Binomial Theorem

Ryan shows off how to estimate asymptotics of strange functions using other functions, by plugging it into software and checking what things converge to.

Sadly Maple is paid software.

But the trick of graphing the ratios and checking what the asymptotic ratios are like looks really useful. I might use Desmos for it, or just plain matplotlib.

Idea for bounding products (e.g. factorials or binomials) is to either put them into a log and then use calculus of choice for estimates, or put upper bounds on individual elements.

For example split a long fraction of format $\frac{a_0 a_1 a_2 a_3 \cdots}{b_0 b_1 b_2 b_3 \cdots}$ into individual fractions and use some other fractions such that $\frac{x}{y} > \frac{a_i}{b_i}$, after that you are safe to take an upper bound. (and vice versa a lower value for lower bounds).

The other idea is to use statistics knowledge, and convert the items we are working with into distributions. For example if $\frac{C_n}{2^n}$ is the proportion of heads in 2^n coin flips, we know using combination of law of large numbers, and standard distribution interpolations that the standard deviation will be $\frac{1}{\sqrt{n}}$.

Lecture 3

Starts with a review of the above street fighting binomials.

lecture 3b

A lot of strategic bounding of the factorial.

Factorial is asymptotic to $\log(n!) \sim n \log n$

You can do $\forall x . n! \geq \frac{x^n}{e^x}$. Best choice is $x = n$ that will be $n! \geq \frac{n^n}{e^n}$, which is a great lower bound.

Next trick is to assume $n! = \frac{n^n}{f(x)}$ then note that $f(x) = \frac{n^n}{n!}$.

Now try $\frac{f(n+1)}{f(n)} = \frac{(n+1)^{n+1} n!}{n^n n! (n+1)}$ where lots of things cancel out to leave $\frac{(n+1)^n}{n^n} = \left(\frac{n+1}{n}\right)^n = \left(1 + \frac{1}{n}\right)^n = e$ which is nice. Now we know $n! = \left(\frac{n^n}{e^n}\right)$

Core ideas here are assume there exists a function representing the ratio of the unknown to a known value, and then put two consequitive terms to let things cancel out. Finally try to get a function for the ratio in terms of n.

Next trick is to $\log(n!) = \log(1) + \log(2) + \ldots + \log(n)$ and then apply an integral here, so $\int_1^0 \log(n) dn = t \log t - t]_{t=1}^n \geq n \log n - n + 1$, important thing is to not get confused by whether you are doing lower and upper bound, and start using overestimating approximations in a lower bound.

Last part of reinman error bounding tricks, which I didn't quite enjoy. Idea is to assume the reinman error triangles are triangles and bound based on that. But apparently that arrives at the Stirling's Formula.

(Unrelated) Conferences

Theoretical:

STOC/FOCS/SODA

Practical:

ESA ALENEX VLDB/SIGMOD

(Unrelated) Lean, Rocq, and Calculus of Constructions

I dedicated a lot of time to trying to understand the calculus of constructions (paper by Helmut Brandl) and the lean4 proof system. However after hours of discussion with Gemini, I couldn't figure out how terms, types, sorts, universes, etc. relate together. It is specifically confusing that sets and standard logic exist here only in a certain strange format.

On a later examination it appears that the rocq docs are quite good compared to lean, though still far of from comprehensible. It appears Rocq uses sets as a basic type, whereas lean doesn't consider sets or zfc to be natural. This makes it quite confusing.

It is however nice to note that haskell and erlang as functional programming languages are quite useful. Though on second examination it appears they have little in common still.

Lecture 3c